



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch62 关系型数据库的价值

Software Institute
Nanjing University
Bei Jia

获取持久化数据

- 持久存储大量数据
- 在大多数的计算架构中，有两个存储区域：
 - 速度快但是数据易丢失的“主存储器”(main memory)
 - ◆ 空间有限
 - ◆ 易挥发
 - 存储量大但速度较慢的“后备存储器”(backing store)
 - ◆ 文件系统，如许多生产力应用程序(productivity application，比如文字处理软件)
 - ◆ 数据库，大多数企业级应用程序

并发

- 多个用户会一起访问同一份数据体，并且可能要修改这份数据。（大多数情况下，他们都在不同数据区域内各自操作，但是，偶尔也会同时操作一小块数据）
- 关系型数据库提供了“事务”机制来控制对其数据的访问，以便处理此问题。
- 事务在处理错误时也有用。通过事务更改数据时，如果在处理变更的过程中出错了，那么就可以回滚（roll back）这一事务，以保证数据不受破坏。

集成

- 企业级应用程序居于一个丰富的生态系统中，它需要与其他应用程序协同工作。不同的应用程序经常要使用同一份数据，而且某个应用程序更新完数据之后，必须让其他应用程序知道这份数据已经改变了。
- 常用的办法是使用共享数据库集成(shared database integration)，多个应用程序都将数据保存在同一个数据库中。这样一来，所有应用程序很容易就能使用彼此的数据了。
- 与多用户访问单一应用程序时一样，数据库的并发控制机制也可以应对多个应用程序。

近乎标准的模型

- 关系型数据库以近乎标准的方式提供了数据模型。
- 尽管各种关系型数据库之间仍有差异，但其核心机制相同
 - 不同厂商的SQL方言相似
 - “事务”的操作方式也几乎一样



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch62 NoSQL的由来

Software Institute
Nanjing University
Bei Jia

阻抗失谐 1

- 基于关系代数(relational algebra), 关系模型把数据组织成“关系”(relation)和“元组”(tuple)。
 - 元组是由“键值对”(name-value pair)构成的集合
 - 而关系则是元组的集合。
 - SQL操作所使用及返回的数据都是“关系”
 - 元组不能包含“嵌套记录”(nested record)或“列表”(list) 等任何结构
- 而内存中的数据结构则无此限制, 它可以使用的数据组织形式比“关系”更丰富。
- 关系模型和内存中的数据结构之间存在差异。这种现象通常称为“阻抗失谐”。
 - 如果在内存中使用了较为丰富的数据结构, 那么要把它保存到磁盘之前, 必须先将其转换成“关系形式。于是就发生了“阻抗失谐”: 需要在两种不同的表示形式之间转译

阻抗失谐 2

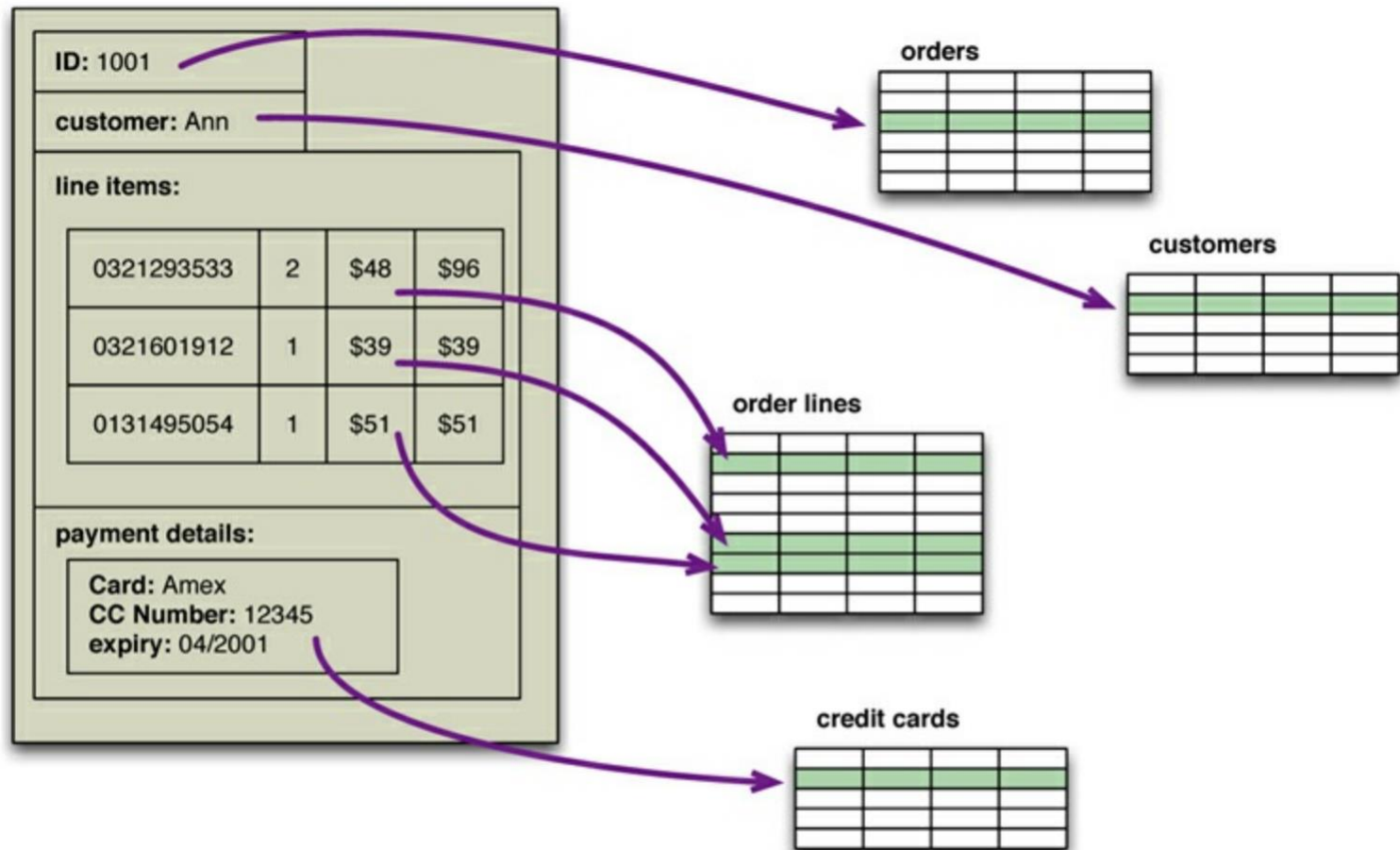


Figure 1.1. An order, which looks like a single aggregate structure in the UI, is split into many rows from many tables in a relational database

解决方法

- 面向对象数据库
- “对象-关系映射框架”(object-relational mapping framework) 通过映射模式 (mapping pattern)表达转换
- 问题:
 - 查询性能问题
 - 集成问题

集成数据库

- SQL充当了应用程序之间的一种集成机制。数据库在这种情况下成了“集成数据库”(integration database)
 - 通常由不同团队所开发的多个应用程序，将其数据存储在一个公用的数据库中。
 - 所有应用程序都在操作内容一致的持久数据，提高了数据通信的效率
 - 为了能将很多应用程序集成起来，数据库的结构比单个应用程序所要用到的结构复杂得多
 - 如果某个应用程序想要修改存储的数据，那么它就得和所有使用此数据库的其他应用程序相协调。
 - 各种应用程序的结构和性能要求不尽相同，数据库通常不能任由应用程序更新其数据。为了保持数据库的完整性，我们需要将这一责任交由数据库自身负责。

应用程序数据库

- 将数据库视为“应用程序数据库”(application database)，其内容只能由一个应用程序的代码库直接访问
 - 由于只有开发应用程序的团队才需要知道其结构，模式的维护与更新就更容易了。由于应用程序开发团队同时管理数据库和应用程序代码，因此可以把维护数据库完整性的工作放在应用程序代码中。
- 交互工作转交由应用程序接口来完成
 - “面向服务架构”、Web服务。使得应用程序间通过平台中立的方式完成集成。
 - 在Web服务作为集成机制后，所交换的数据可以拥有更为灵活的结构
 - ◆ 如XML、JSON格式，它们均能够使用嵌套记录及列表等更丰富的数据结构
 - ◆ 使用“面向文档”的交互方式，减少通讯次数和开销
 - ◆ 既可以传输文本，也可以传输二进制
- 在使用应用程序数据库后，由于内部数据库与外部通信服务之间已经解耦，所以外界并不关心数据如何存储，这样就可以选用非关系型数据库了
 - 关系型数据库的许多特性，诸如安全性等，可以交给使用该数据库的外围应用程序(enclosing application)来做

集群问题

- 纵向扩展(scale up)及横向扩展(scale out)
 - 采用集群应对横向扩展
- 关系型数据库的“分片”和“复制”
 - 在负载分散的同时，应用程序必须控制所有分片，需要知道数据库中的每份小数据的存储情况
 - 如何确保跨分片的查询、参照完整性(referential integrity)、事务、一致性控制(consistency control)等操作

NoSQL

- NoSQL没有规范的定义
 - “开源分布式的非关系型数据库”
- 各种NoSQL数据库的共同特性是
 - 不使用关系模型
 - 在集群中运行良好
 - ◆ 关系型数据库使用ACID事务来保持整个数据库的一致性，而这种方式本身与集群环境相冲突
 - ◆ NoSQL数据库为处理并发及分布问题提供了众多选项。
 - 开源
 - 适用于21世纪的互联网公司
 - 无模式
 - ◆ 不用事先修改结构定义，即可自由添加字段了
 - ◆ 这在处理不规则数据和自定义字段时非常有用



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch64 聚合

Software Institute
Nanjing University
Bei Jia

聚合

- 把一组相互关联的对象视为一个整体单元来操作，而这个单元就叫聚合（aggregate）。
 - 通过原子操作(atomic operation)更新聚合的值（含一致性管理）
 - 以聚合为单位与数据存储通信
 - 在集群中操作数据库时，用聚合为单位来复制和分片
 - 由于程序员经常通过聚合结构来操作数据，故而采用聚合也能让其工作更为轻松。
- 面向聚合操作数据时所用的单元，其结构比元组集合复杂得多
 - “键值数据库”、“文档数据库”、“列族数据库”

关系模型

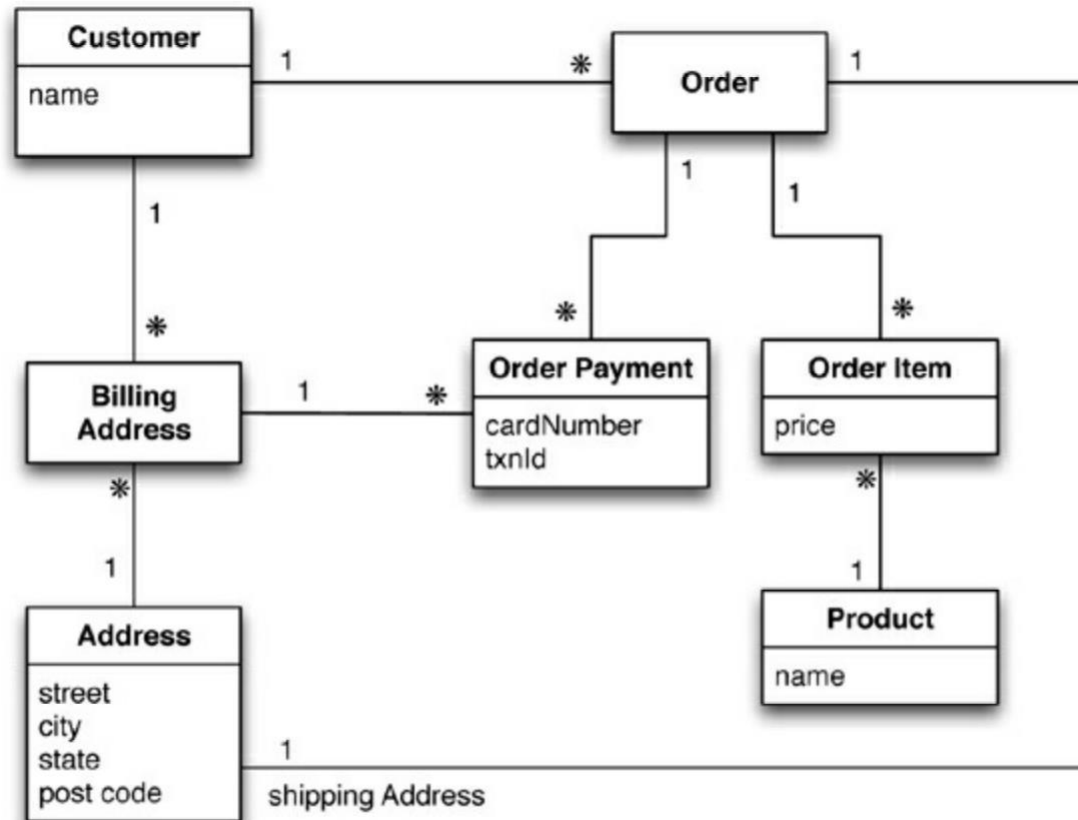


Figure 2.1. Data model oriented around a relational database (using UML notation [\[Fowler UML\]](#))

关系实例

Customer	
Id	Name
1	Martin

Orders		
Id	CustomerId	ShippingAddressId
99	1	77

Product	
Id	Name
27	NoSQL Distilled

BillingAddress		
Id	CustomerId	AddressId
55	1	77

OrderItem			
Id	OrderId	ProductId	Price
100	99	27	32.45

Address	
Id	City
77	Chicago

OrderPayment				
Id	OrderId	CardNumber	BillingAddressId	txnId
33	99	1000-1000	55	abelif879rft

Figure 2.2. Typical data using RDBMS data model

聚合数据模型

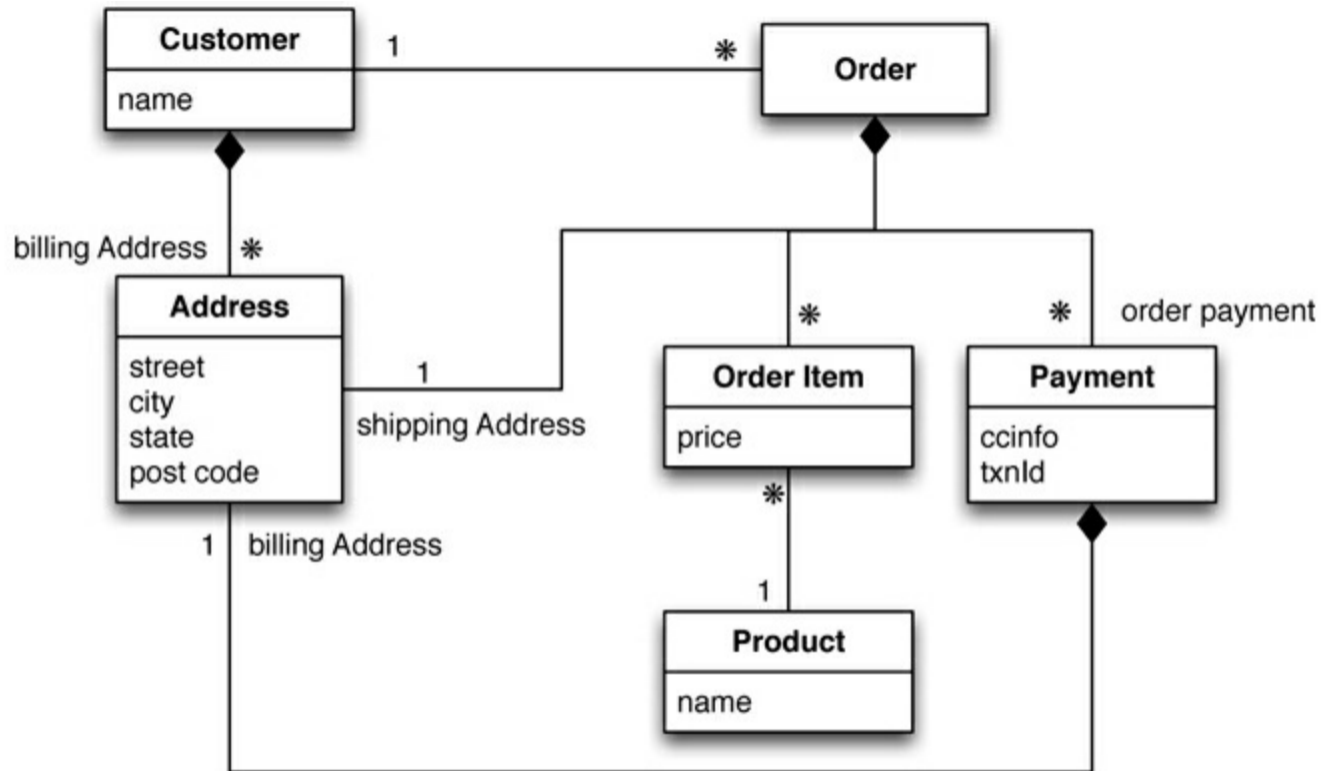


Figure 2.3. An aggregate data model

聚合实例（两个聚合）

```
// in customers
{
  "id":1,
  "name":"Martin",
  "billingAddress":[{"city":"Chicago"}]
}

// in orders
{
  "id":99,
  "customerId":1,
  "orderItems":[
    {
      "productId":27,
      "price": 32.45,
      "productName": "NoSQL Distilled"
    }
  ],
  "shippingAddress":[{"city":"Chicago"}]
  "orderPayment":[
    {
      "ccinfo":"1000-1000-1000-1000",
      "txnId":"abelif879rft",
      "billingAddress": {"city": "Chicago"}
    }
  ],
}
```

另一种聚合

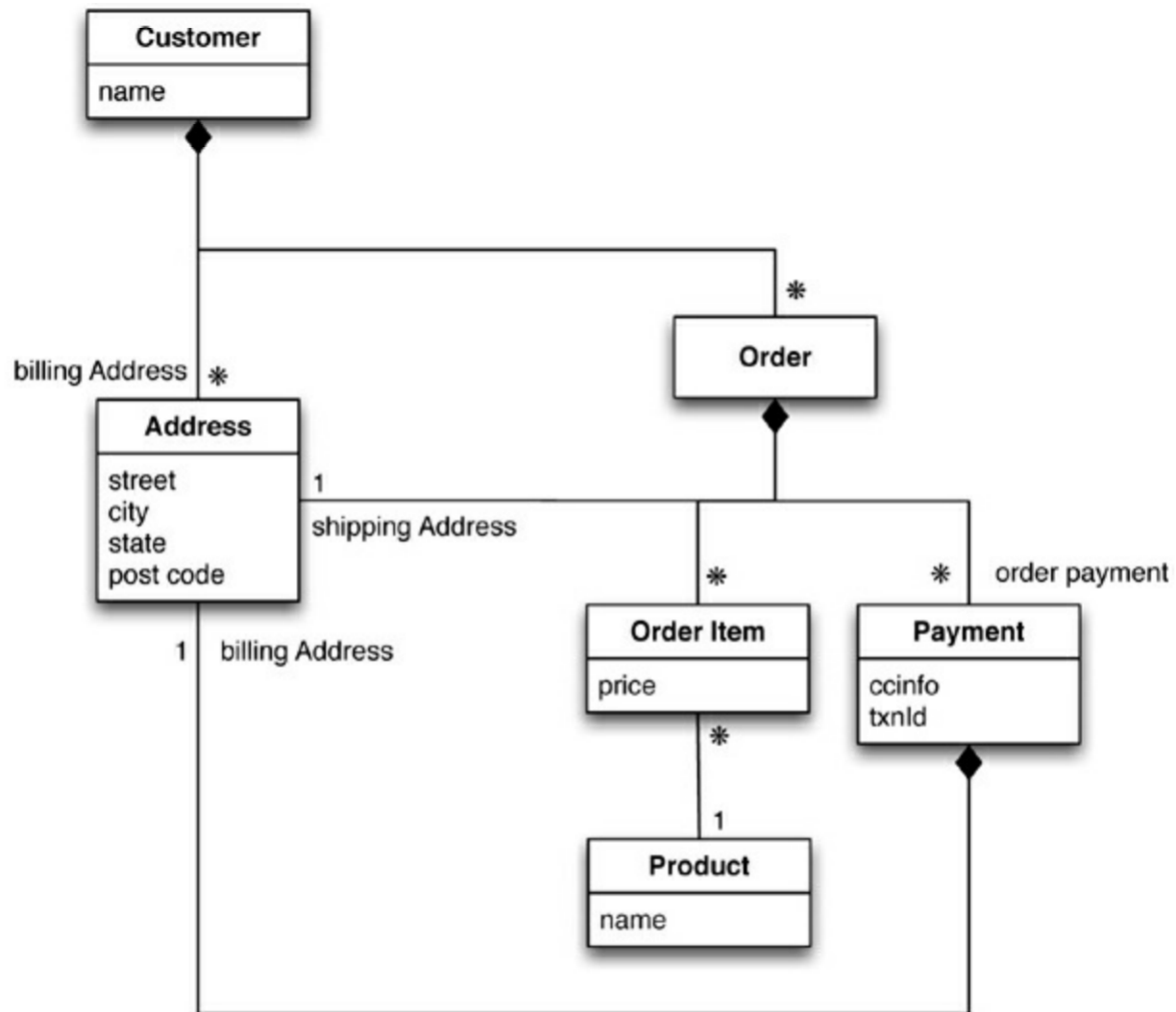


Figure 2.4. Embed all the objects for customer and the customer's orders

聚合实例（一个聚合）

```
// in customers
{
  "customer": {
    "id": 1,
    "name": "Martin",
    "billingAddress": [{"city": "Chicago"}],
    "orders": [
      {
        "id": 99,
        "customerId": 1,
        "orderItems": [
          {
            "productId": 27,
            "price": 32.45,
            "productName": "NoSQL Distilled"
          }
        ],
        "shippingAddress": [{"city": "Chicago"}]
      },
      {
        "id": 100,
        "customerId": 1,
        "orderItems": [
          {
            "productId": 28,
            "price": 19.99,
            "productName": "NoSQL Distilled"
          }
        ],
        "shippingAddress": [{"city": "Chicago"}]
      }
    ],
    "orderPayment": [
      {
        "ccinfo": "1000-1000-1000-1000",
        "txnId": "abelif879rft",
        "billingAddress": {"city": "Chicago"}
      },
      {
        "ccinfo": "1000-1000-1000-1000",
        "txnId": "abelif879rft",
        "billingAddress": {"city": "Chicago"}
      }
    ]
  }
}
```

聚合无知

- 关系型数据库的数据模型中，没有“聚合”这一概念，因此我们称之为“聚合无知”(aggregate- ignorant)。
 - “图数据库”也是聚合无知的。
- 聚合反应数据操作的边界，很难在共享数据的多个场景中“正确”划分，对某些数据交互有用的聚合结构，可能会阻碍另一些数据交互
 - 在客户下单并核查订单，以及零售商处理订单时，将订单视为一个聚合结构就比较合适。
 - 如零售商要分析过去几个月的产品销售情况，那么把订单做成一个聚合结构反而麻烦了。要取得商品销售记录，就必须深挖数据库中的每一个聚合。
- 若是采用“聚合无知模型”，那么很容易就能以不同方式来查看数据
 - 在操作数据时，如果没有一种占主导地位的结构，那么选用此模型效果会更好。

聚合之间的关系

- 例如：把订单和客户放在两个聚合中，但是想在它们之间设定某种关系，以便能根据订单查出客户数据。
 - 要提供这种关联，最简单的办法就是把客户ID嵌入订单的聚合数据中。在应用层级提供关联。
 - 在数据库层级提供聚合之间关系的表达机制
- 操作多个有关联的聚合，由应用保证其正确性
 - 面向聚合数据库获取数据时以聚合为单元，只能保证单一聚合内部内容的原子性。

聚合、集群和事务处理

- 在集群上运行时，需要把采集数据时所需的节点数降至最小
 - 如果在数据库中明确包含聚合结构，那么它就可以根据这一重要信息，知道哪些数据需要一起操作了，而且这些数据应该放在同一个节点中
- 通常情况下，面向聚合的数据库不支持跨越多个聚合的ACID事务。它每次只能在一个聚合结构上执行原子操作。
 - 如果想以原子方式操作多个聚合，那么就必须自己组织应用程序的代码
 - 在实际应用中，大多数原子操作都可以局限于某个聚合结构内部，而且，在将数据划分为聚合时，这也是要考虑的因素之一



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch65 主要的NoSQL数据模型

Software Institute
Nanjing University
Bei Jia

键值数据模型与文档数据模型

- 这两类数据库都包含大量聚合，每个聚合中都有一个获取数据所用的键或ID。
- 两种模型的区别是：
 - 键值数据库的聚合不透明，只包含一些没有太多意义的大块信息
 - ◆ 聚合中可以存储任意数据。数据库可能会限制聚合的总大小，但除此之外，其他方面都很随意
 - ◆ 在键值数据库中，要访问聚合内容，只能通过键来查找
 - 在文档数据库的聚合中，可以看到其结构。
 - ◆ 限制其中存放的内容，它定义了其允许的结构与数据类型
 - ◆ 能够更加灵活地访问数据。通过用聚合中的字段查询，可以只获取一部分聚合，而不用获取全部内容
 - ◆ 可以按照聚合内容创建索引

列族存储 1

- 大部分数据库都以行为单元存储数据。然而，有些情况下写入操作执行得很少，但是经常需要一次读取若干行中的很多列。此时，列存储数据库将所有行的某一组列作为基本数据存储单元
- 列族数据库将列组织为列族。每一列都必须是某个列族的一部分，而且访问数据的单元也得是列
 - 某个列族中的数据经常需要一起访问。
- 列族模型将其视为两级聚合结构(two-level aggregate structure)。
 - 与“键值存储”相同，第一个键通常代表行标识符，可以用它来获取想要的聚合。
 - 列族结构与“键值存储”的区别在于，其“行聚合”(row aggregate)本身又是一个映射，其中包含一些更为详细的值。这些“二级值”(second-level value)就叫做“列”。与整体访问某行数据一样，我们也可以操作特定的列。

列族存储 2

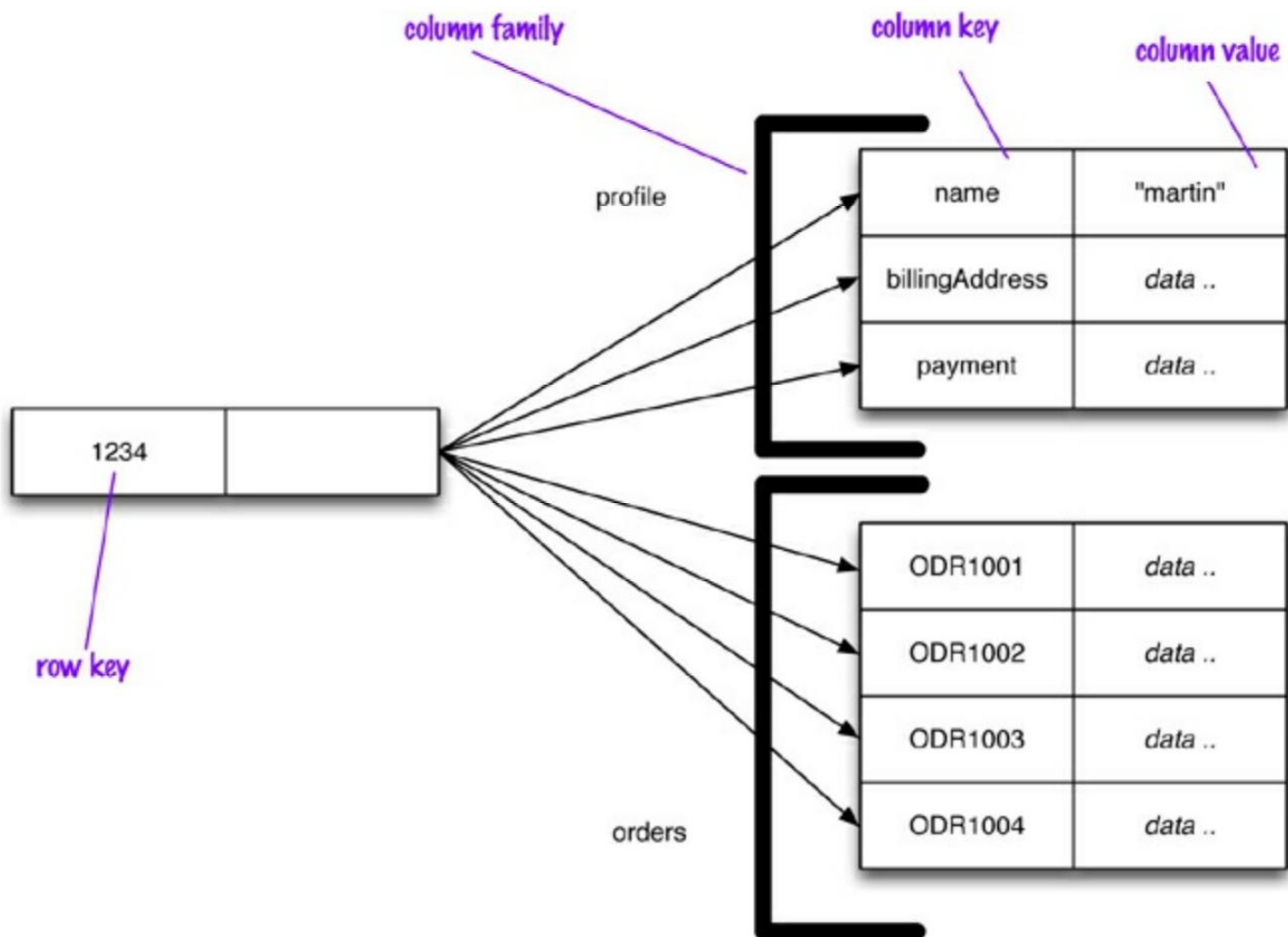


Figure 2.5. Representing customer information in a column-family structure

列族存储 3

- 两种数据组织方式

- 面向行(row-oriented): 每一行都是一个聚合(例如ID为1234的顾客就是一个聚合), 该聚合内部存有一些包含有用数据块(客户信息、订单记录)的列族
- 面向列(column-oriented): 每个列族都定义了一种记录类型(例如客户信息), 其中每行都表示一条记录。数据库中的大“行”理解为列族中每一个短行记录的串接

面向聚合的数据模型

- 共同点

- 都使用聚合这一概念，而且聚合中都有一个可以查找其内容的索引键。
- 在集群上运行时，聚合是中心环节，因为数据库必须保证将聚合内的数据存放在同一个节点上。
- 聚合是“更新”操作的最小数据单位(atomic unit)，对事务控制来说，以聚合为操作单元

- 差别

- 键值数据模型将聚合看作不透明的整体，只能根据键来查出整个聚合，而不能仅仅查询或获取其中的一部分
- 文档模型的聚合对数据库透明，于是就可以只查询并获取其中一部分数据了，不过，由于文档没有模式，因此在想优化存储并获取聚合中的部分内容时，数据库不太好调整文档结构
- 列族模型把聚合分为列族，让数据库将其视为行聚合内的一个数据单元。此类聚合的结构有某种限制，但是数据库可利用此种结构的优点来提高其易访问性。

图结构

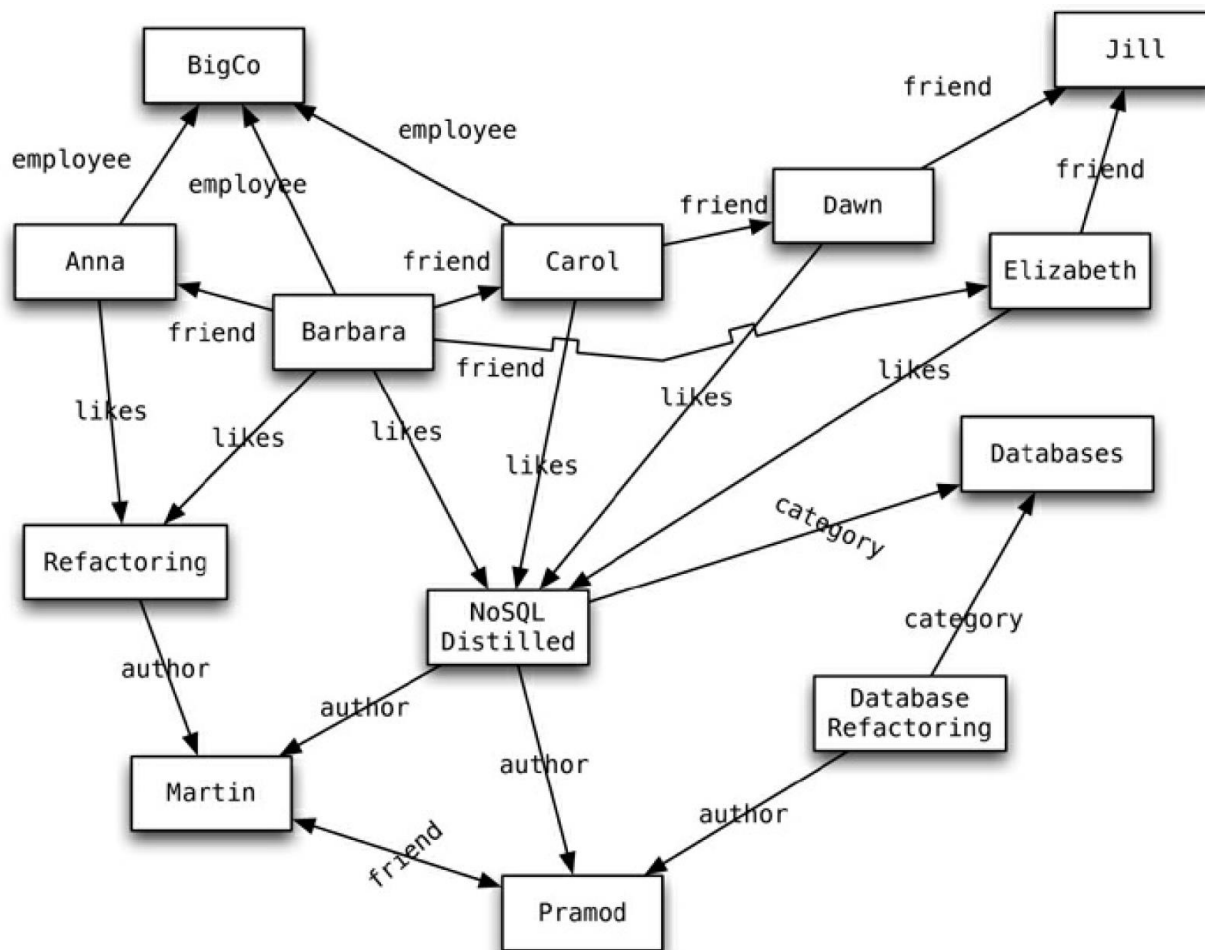


Figure 3.1. An example graph structure

图数据库

- 图数据库的基本数据模型：由边(或称“弧”，arc)连接而成的若干节点。
- 可以用专门为“图”而设计的查询操作来搜寻图数据库的网络了
 - 指定节点，通过边进行查询
- 关系型数据可以通过“外键”实现，查询中的多次连接，效率较差

图数据库

- 图数据库的基本数据模型：由边(或称“弧”，arc)连接而成的若干节点。
- 可以用专门为“图”而设计的查询操作来搜寻图数据库的网络了
 - 指定节点，通过边进行查询
- 关系型数据可以通过“外键”实现，查询中的多次连接，效率较差

无模式

- 关系型数据库中，首先必须定义“模式”，然后才能存放数据。
- NoSQL数据库，无模式：
 - “键值数据库”可以把任何数据存放在一个“键”的名下。
 - “文档数据库”对所存储的文档结构没有限制
 - 在列族数据库中，任意列里面都可以随意存放数据
 - 图数据库中可以新增边，也可以随意向节点和边中添加属性。

格式不一致的数据

- 每条记录都拥有不同字段集(set of field)
- 关系型数据库中，“模式”会将表内每一行的数据类型强行统一，若不同行所存放的数据类型不同，那这么做就很别扭。
 - 要么得分别用很多列来存放这些数据，而且把用不到的字段值填成null(这就成了“稀疏表”， sparse table)，
 - 要么就要使用类似custom column 4这样没有意义的列类型。
- 无模式表则没有这么麻烦，每条记录只要包含其需要的数据即可，不用再担心上面的问题了。

无模式的问题

- 存在“隐含模式”。在编写数据操作代码时，对数据结构所做的一系列假设
 - 应用与数据的耦合问题
 - 无法在数据库层级优化和验证数据
- 在集成数据库中，很难解决
 - 使用应用程序数据库，并使用Web Services、SOA等完成集成
 - 在聚合中为不同应用程序明确划分出不同区域
 - ◆ 在文档数据库中，可以把文档分成不同的区段(section)
 - ◆ 在列族数据库，可以把不同的列族分给不同的应用程序



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch66 分布式模型

Software Institute
Nanjing University
Bei Jia

数据分布

- 数据分布有两条路径:分片(sharding)与复制(replication)。
 - “分片”则是将不同数据存放在不同节点中
 - “复制”就是将同一份数据拷贝至多个节点;
 - ◆ “主从式”(master-slave)和“对等式”(peer-to-peer)
- 既可以在两者中选一个来用,也可以同时使用它们。

单一服务器

- 最简单的分布形式：根本不分布。
 - 将数据库放在一个节点中，让它处理对数据存储的读取与写入操作。
 - 不用考虑使用其他方案时所需应对的复杂事务，这对数据操作管理者与应用程序开发者来说，都比较简单。
- 尽管许多NoSQL数据库都是为集群运行环境而设计的，但是只要符合应用程序需求，那就完全可以按照单一服务器的分布模型来使用
 - 图数据库配置在一台服务器上
 - 如果只是为了处理聚合，那么可以考虑在单一服务器上部署“文档数据库”或“键值数据库”

分片 1

- 一般来说，数据库的繁忙体现在：不同用户需要访问数据集中的不同部分。
- 在这种情况下，把数据的各个部分存放于不同的服务器中，以此实现横向扩展。该技术就叫“分片”(sharding)。

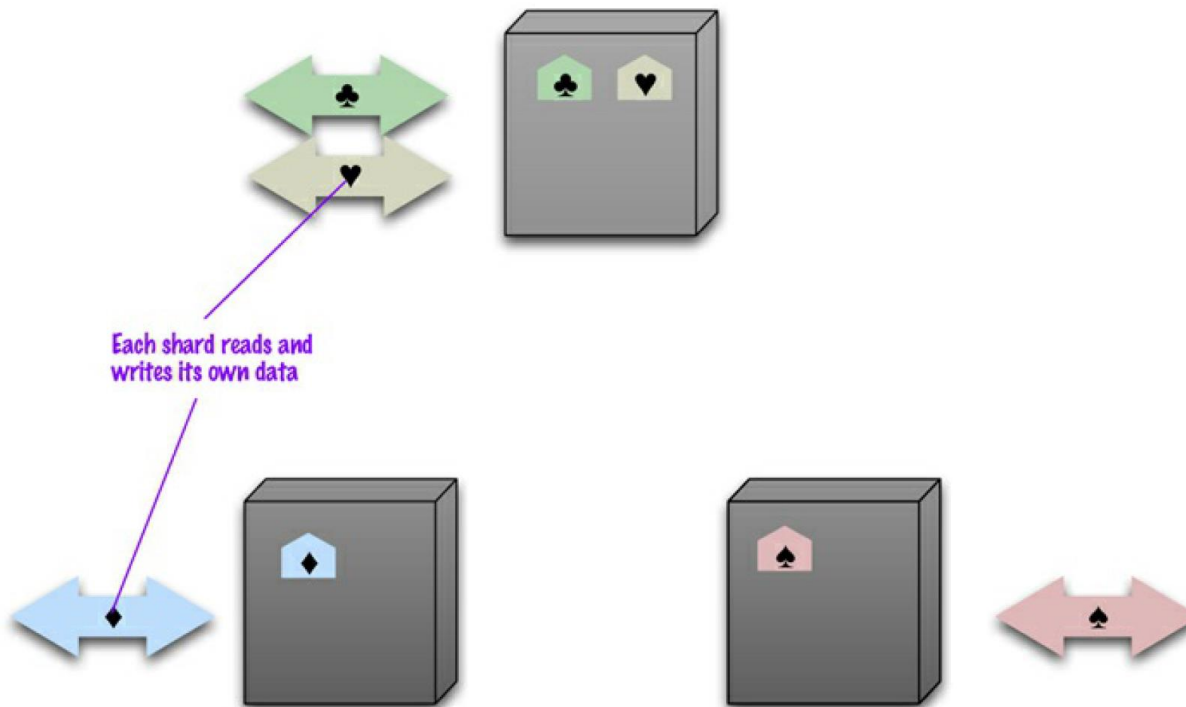


Figure 4.1. Sharding puts different data on separate nodes, each of which does its own reads and writes.

分片 2

- 在理想情况下，不同的服务器节点会服务于不同的用户。每位用户只需与一台服务器通信，并且很快就能获得服务器的响应。网络负载相当均衡地分布于各台服务器上。
- 为达成目标，必须保证需要同时访问的那些数据都存放在同一节点上，而且节点必须排布好这些数据块，使访问速度最优。
 - 若使用面向聚合的数据库，可以把聚合作为分布数据的单元。
 - 在节点的数据排布问题上，有若干个与性能改善相关的因素。
 - ◆ 地理因素
 - ◆ 负载均衡
 - ◆ 聚合有序放置

分片 3

- 采用应用程序的逻辑实现分片
 - 编程模型复杂化，因为应用程序的代码必须负责把查询操作分布到多个分片上
 - 若想重新调整分片，那么既要修改程序代码，又要迁移数据
- 采用NoSQL数据库提供的“自动分片”(auto-sharding)功能
 - 让数据库自己负责把数据分布到各分片
 - 并且将数据访问请求引导至适当的分片上

分片 4

- 分片可以同时提升读取与写入效率
 - 使用“复制”技术，尤其是带缓存的复制，可以极大地改善读取性能，但对于写操作帮助不大
- 分片对改善数据库的“故障恢复能力”帮助并不大。尽管数据分布在不同的节点上，但是和“单一服务器”方案一样，只要某节点出错，那么该分片上的数据就无法访问了
 - 在发生故障时，只有访问此数据的那些用户才会受影响，而其余用户则能正常访问
 - 由于多节点问题，从实际效果出发，分片技术可能会降低数据库的错误恢复能力

主从复制 1

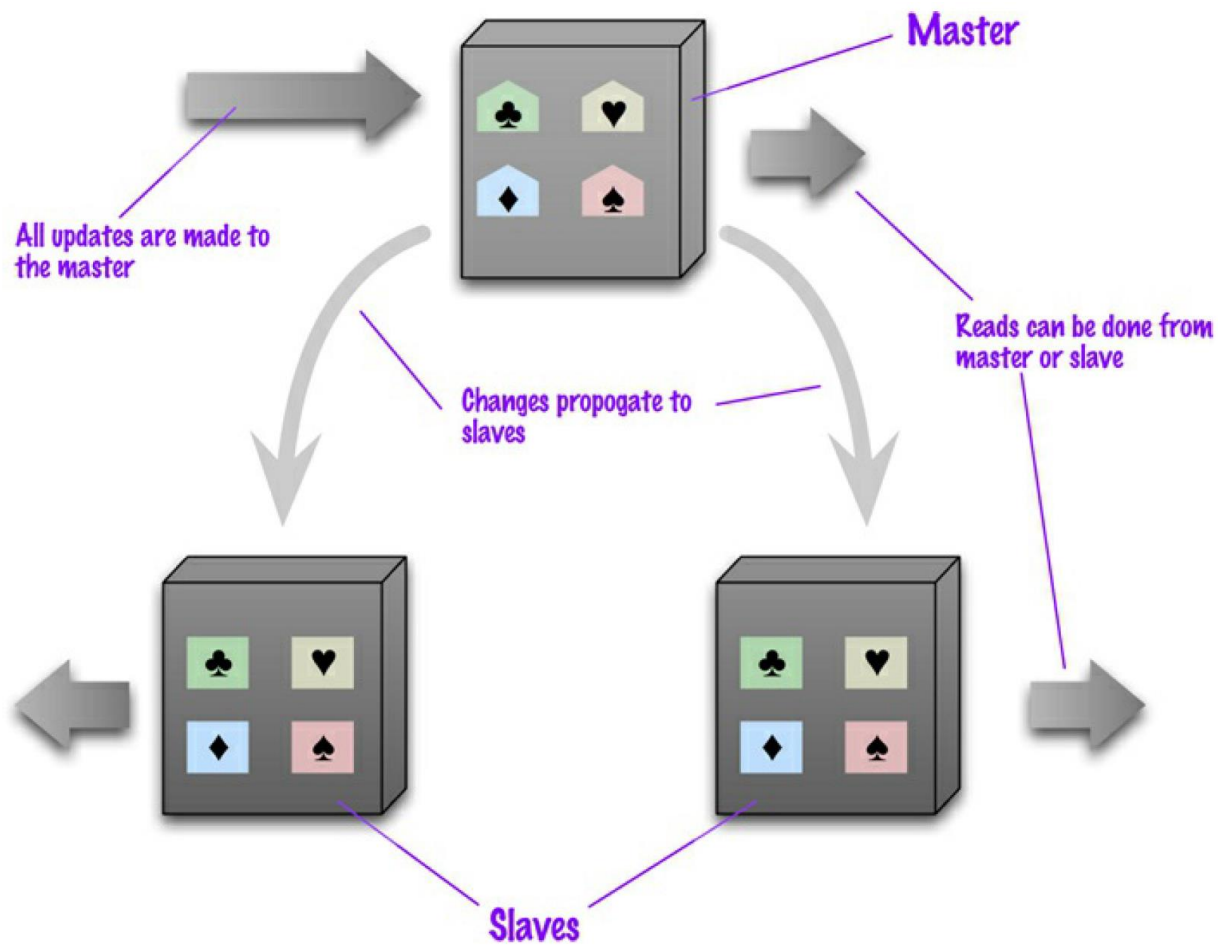


Figure 4.2. Data is replicated from master to slaves. The master services all writes; reads may come from either master or slaves.

主从复制 2

- 在“主从式分布”(master-slave distribution)中
 - 其中有一个节点叫做“主(master) 节点”，或“主要(primary) 节点”。主节点存放权威数据，而且通常负责处理数据更新操作。
 - 其余节点都叫“从(slave) 节点”，或“次要(secondary) 节点”，和主节点保持同步，负责读取操作。
- 在需要频繁读取数据集的情况下，“主从复制”(master-slave replication) 有助于提升数据访问性能
 - 以新增更多从节点的方式来进行水平扩展，就可以同时处理更多数据读取请求，并且能保证将所有请求都引导至从节点
 - 在写入操作特别频繁的场所，数据库仍受制于主节点处理更新，以及向从节点发布更新的能力
- “主从复制”可以增强“读取操作的故障恢复能力”(read resilience)
 - 万一主节点出错了，那么从节点依然可以处理读取请求。
 - 主节点出错之后，除非将其恢复，或另行指派新的主节点，否则数据库就无法处理写入操作。
 - 在主节点出错之后，由于拥有内容与主节点相同的从节点，很快就能指派一个从节点作为新的主节点，从而具备故障恢复能力。
 - 主节点可以手工指派，也可自动选择。
- “数据的不一致性”

对等复制

- “对等复制” 它没有“主节点”这一概念。所有“副本”(replica) 地位相同，都可以接受写入请求，而且丢失其中一个副本，并不影响整个数据库的访问。

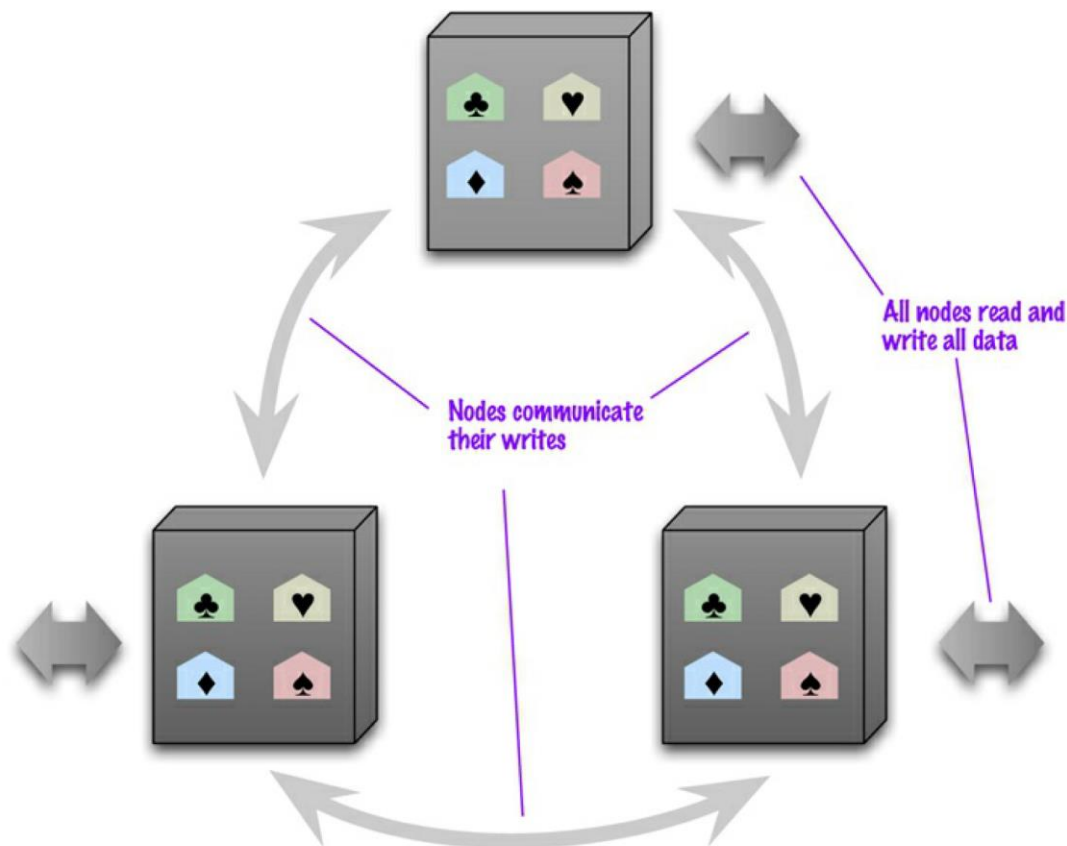


Figure 4.3. Peer-to-peer replication has all nodes applying reads and writes to all the data.

结合“主从复制”与“分片”

- 如果同时使用“主从复制”与“分片”，那么就意味着整个系统有多个主节点，然而对每项数据来说，负责它的主节点只有一个
- 根据配置需要，同一个节点既可以做某些数据的主节点，也可以充当其他数据的从节点，此外，也可以指派全职的主节点或从节点

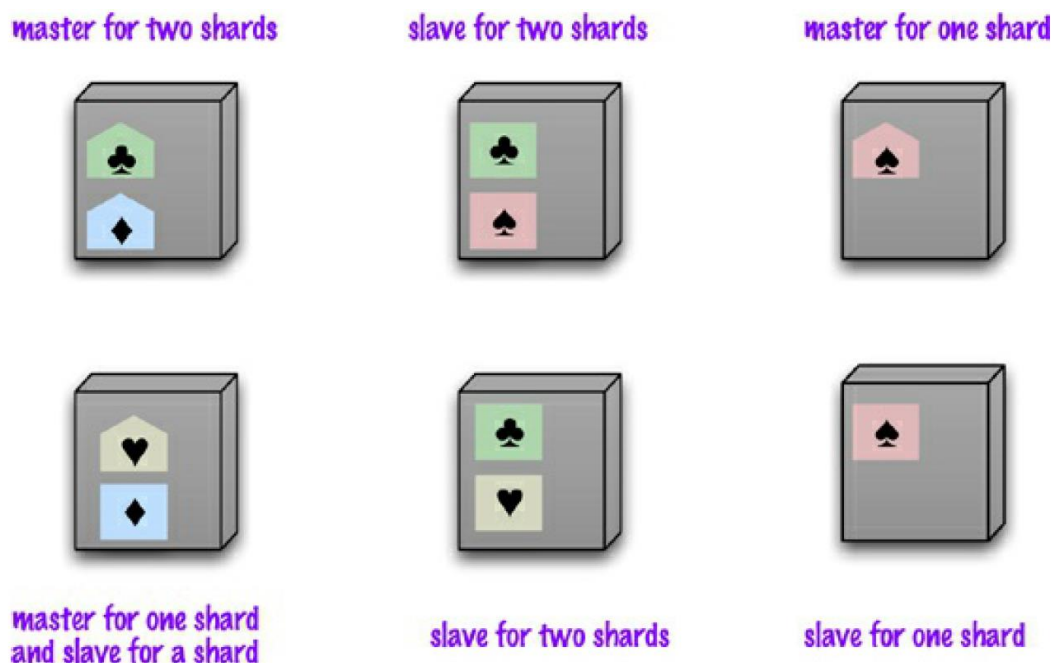


Figure 4.4. Using master-slave replication together with sharding

结合“对等复制”与“分片”

- 使用列族数据库时，经常会将“对等复制”与“分片”结合起来。
- 数据可能分布于集群中的数十个或数百个节点上。在采用“对等复制”方案时，一开始可以用“3”作为复制因子(replication factor), 也就是把每个分片数据放在3个节点中。一旦某个节点出错，那么它上面保存的那些分片数据会由其他节点重建

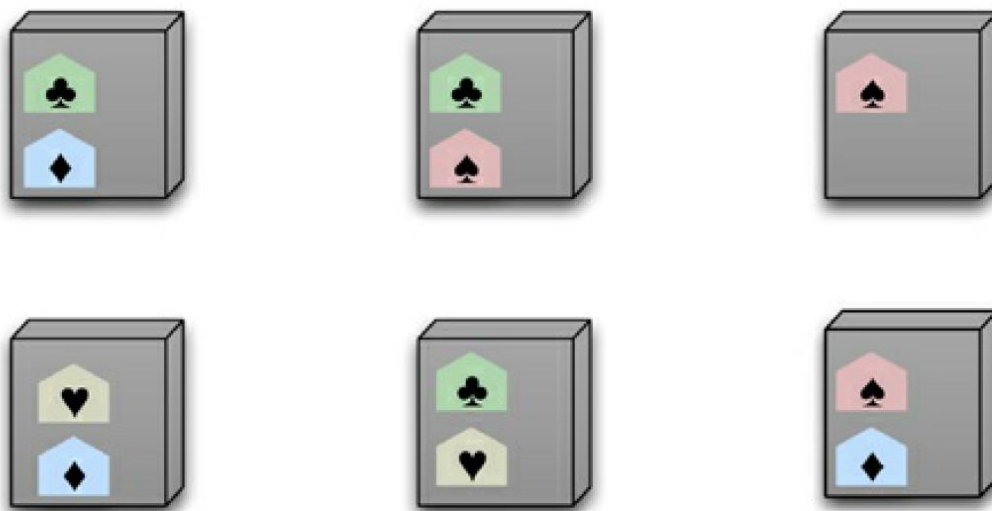


Figure 4.5. Using peer-to-peer replication together with sharding



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch67 分布式模型中的一致性

Software Institute
Nanjing University
Bei Jia

写入冲突和读写冲突

- 当两个客户端试图同时修改一份数据时，会发生“写入冲突”。而当某客户端在另一个客户端执行写入操作的过程中读取数据时，则会发生“读写冲突”。
- 悲观方式以锁定数据记录来避免冲突
 - “写入锁” (write lock)
- 乐观方式则在事后检测冲突并将其修复
 - “条件更新”(conditional update)，任意客户在执行更新操作之前，都要先测试数据的当前值和其上一次读入的值是否相同
 - 保存冲突数据。用户自行“合并”(merge)或“自动合并”（面向特定领域)

NoSQL的不一致性

- “图数据库”常常和关系型数据库一样，也支持ACID事务。
- 面向聚合的数据库通常支持“原子更新”(atomic update),但仅限于单一聚合内部
 - “一致性”可以在某个聚合内部保持，但在各聚合之间则不行
 - 在执行影响多个聚合的更新操作时，会留下一段时间空档，让客户端有可能在此刻读出逻辑不一致的数据
 - 存在不一致风险的时间长度就叫“不一致窗口”(inconsistency window)。

复制一致性 1

- “复制一致性”(replication consistency)。要求从不同副本中读取同一个数据项时，所得到的值相同

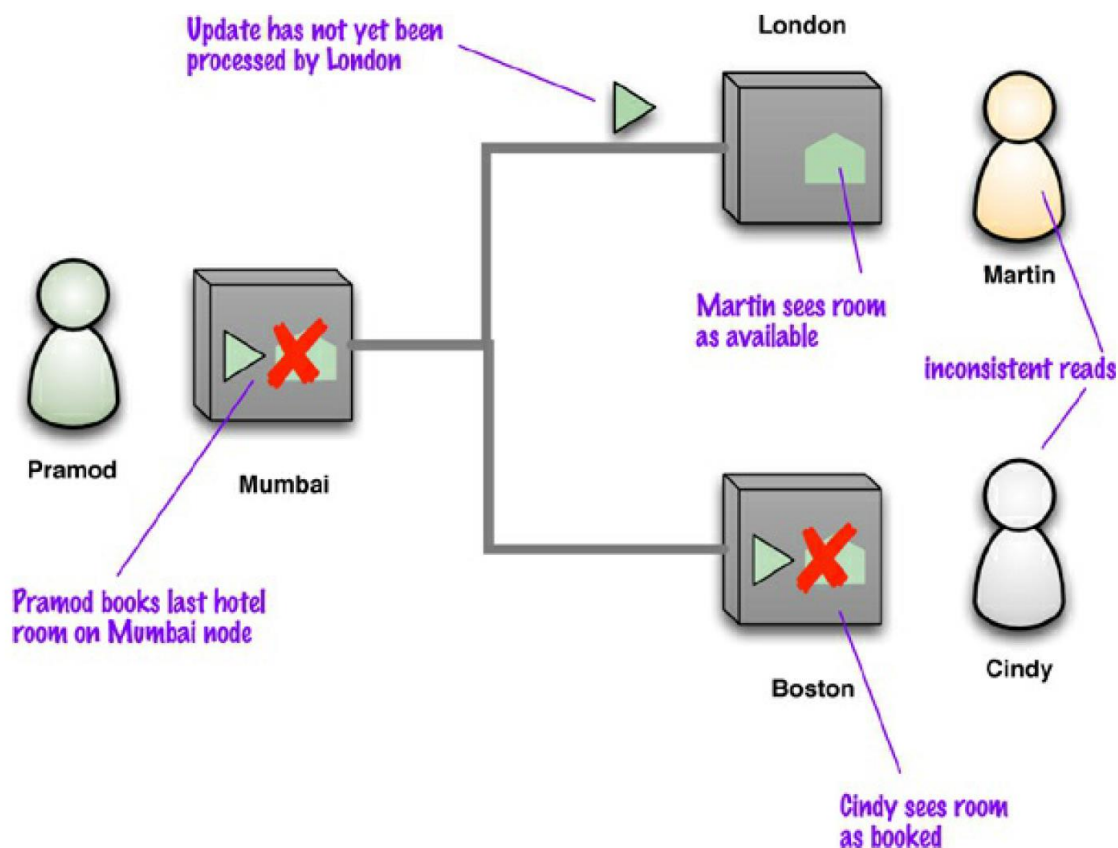


Figure 5.2. An example of replication inconsistency

复制一致性 2

- 在分布式系统中，如果某些节点收到了更新数据，而另外一些节点却尚未收到，那么这种情况就视为“读写冲突”。
- 若写入操作已经传播至所有节点，则此刻的数据库就具备“最终一致性”(eventually consistent)
- 复制不一致性带来的“不一致窗口”，在考虑网络环境后，会比单一节点导致的“不一致窗口”长的多
 - 不一致性窗口对应用的影响不同

照原样读出所写内容的一致性 1

- “照原样读出所写内容的一致性”(read-your-writes consistency) , 在执行完更新操作之后, 要能够立刻看到新值。
- 在具备“最终一致性”的系统中, 可以提供“会话一致性”(session consistency) :在用户会话内部保持“照原样读出所写内容的一致性”
 - 使用“黏性会话”(sticky session), 即绑定到某个节点的会话(这种性质也叫做“会话亲和力”, session affinity).
 - ◆ “黏性会话”可以保证, 只要某节点具备“照原样读出所写内容的一致性”, 那么与之绑定的会话就都具备这种特性了。
 - ◆ “黏性会话”的缺点是, 它会降低“负载均衡器”(load balancer)的效能
 - 使用“版本戳”(version stamp), 并确保同数据库的每次交互操作中, 都包含会话所见的最新版本戳。服务器节点在响应请求之前必须先保证, 它所含有的更新数据包含此版本戳。

分布式系统中的一致性

- 使用“黏性会话”和“主从复制”来保证“会话一致性”时，由于读取与写入操作分别发生在不同节点，那么想保证这一点会比较困难。
 - 方法一：将写入请求先发给从节点，由它负责将其转发至主节点，并同时保持客户端的“会话一致性”。
 - 方法二：在执行写入操作时临时切换到主节点，并且在从节点尚未收到更新数据的这一段时间内，把读取操作都交由主节点来处理。



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch68 放宽“一致性”和“持久性”约束

Software Institute
Nanjing University
Bei Jia

使用事务保障“一致性”

- 使用“事务”达成强一致性
- 引入放松“隔离级别” (isolation level)的功能，以允许查询操作读取尚未提交的数据。
 - 读未提交，一个事务可以读取另一个未提交事务的数据。脏读
 - 读已提交，一个事务要等另一个事务提交后才能读取数据。不可重复读
 - 可重复读，在开始读取数据（事务开启）时，不再允许修改操作。幻读
 - 可串行化，事务串行化顺序执行。严格一致性，效率是一个问题

事务的问题

- 在并发不大的前提下，是否需要事务
- 在数据较多的情况下，为了让应用性能符合用户要求，它们必须弃用“事务”
 - 尤其在需要引入分片机制时，更是如此
- 在分布式应用中，如事务的业务范围涉及多个以网络连接的参与者。其规模、复杂度和波动性均导致无法使用事务进行良好描述

CAP定理

- CAP定理:给定“一致性”(Consistency)、“可用性”(Availability)、“分区耐受性”(Partition tolerance)这三个属性，我们只能同时满足其中两个属性。
 - “一致性”
 - “可用性”，如果客户可以同集群中的某个节点通信，那么该节点就必然能够处理读取及写入操作。
 - “分区耐受性”，如果发生通信故障，导致整个集群被分割成多个无法互相通信的分区时(这种情况也叫“脑裂”，split brain)，集群仍然可用。

“脑裂”的例子

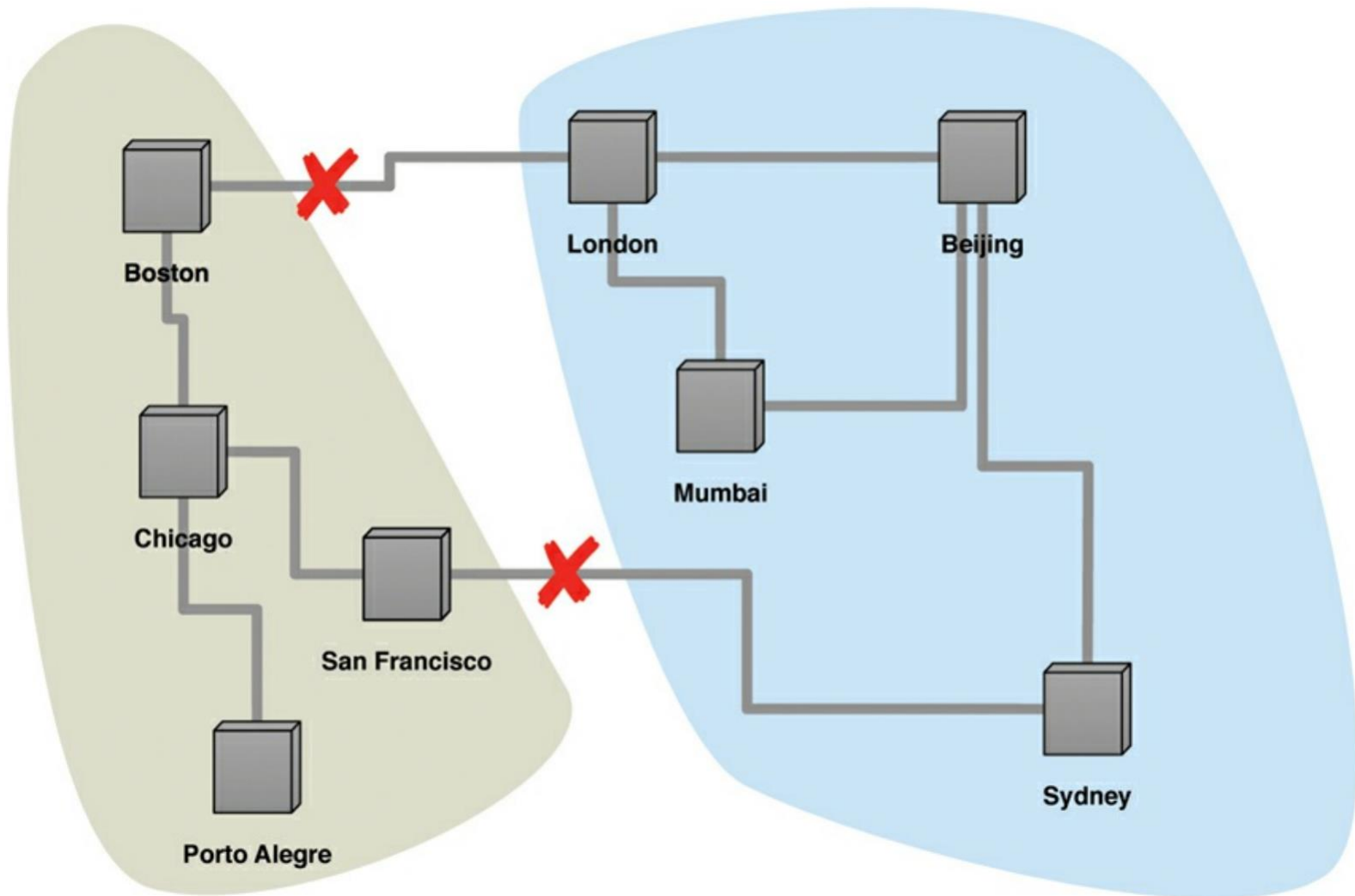


Figure 5.3. With two breaks in the communication lines, the network partitions into two groups.

CA系统

- CA系统，也就是具备“一致性”(Consistency)与“可用性”(Availability)， 但却不具备“分区耐受性”的系统
 - 大多数关系型数据库
- CA集群
 - 无法保证“分区耐受性”，这使得一旦“分区”发生，所有节点必须停止运作
 - CAP中的，可用性定义为“系统中某个无故障节点所接收的每一条请求，无论成功或失败，都必将得到响应。”
 - 介于此时所有节点均为故障节点，不违反CAP中的“可用性”

CAP定理的现实含义

- 尽管“CAP定理”经常表述为“三个属性中只能保有二个”，实际上当系统可能会遭遇“分区”状况时(比如分布式系统)，需要在“一致性”与“可用性”之间进行权衡。
 - 这并不是个二选一的决定，通常来说，我们都会略微舍弃“一致性”，以获取某种程度的“可用性”
 - 这样的系统，既不具备完美的“一致性”，也不具备完美的“可用性”
 - 但是能够满足需要

缺乏“可用性”的例子

- 假设Martin与Pramod都想预订某旅馆的最后一间客房，预订系统使用“对等式分布模型”，它由两个节点组成
 - Martin 使用位于伦敦的节点，而Pramod使用位于孟买的节点。
 - 若要确保一致性，那么当Martin要通过位于伦敦的节点预订房间时，该节点在确认预订操作之前，必须先告知位于孟买的节点。
 - 两个节点必须按照相互一致的顺序来处理它们所收到的操作请求
- 此方案保证了“一致性”，但是假如网络连接发生故障，那么由故障导致的两个“分区”系统，就都无法预订旅馆房间了，于是系统失去了“可用性”。

改善“可用性”的例子

- 指派其中一个节点作为某家旅馆的“主节点”，确保所有预订操作都由“主节点”来处理。
 - 假设位于孟买的节点是“主节点”，那么在两个节点之间的网络连接发生故障之后，它仍然可以处理该旅馆的房间预订工作，这样 Pramod 将会订到最后一间客房
 - 位于伦敦的用户看到的房间剩余情况会与孟买不一致，但是他们无法预订客房，于是就出现了“更新不一致”现象。
 - Martin 可以和位于伦敦的节点通信，但是该节点却无法更新数据。于是出现了“可用性”故障(availability failure)
- 这种在“一致性”与“可用性”之间所做的权衡，能正确处理上述特殊状况。

进一步改善“可用性”的例子

- 让两个“分区”系统都接受客房预订请求，即使在发生网络故障时也如此。
 - 这种方案的风险是，Martin和Pramod有可能都订到了最后一间客房。然而，根据这家旅馆的具体运营情况，这也许不会出问题：
 - ◆ 通常来说，旅行公司都允许一定数量的超额预订，这样的话，如果有某些客人预订了房间而最终没有人住，那么就可以把这部分空余房间分给那些超额预订的人了
 - ◆ 与之相对，某些旅馆总是会在全部订满的名额之外多留出几间客房，这样万一哪间客房出了问题，或者在房间订满之后又来了一位贵宾，那么旅馆可以把客人安排到预留出来的空房中
 - ◆ 还有些旅馆甚至选择在发现预订冲突之后向客户致歉并取消此预订。
- 该方案所付出的代价，要比因为网络故障而彻底无法预订的代价小。

一个写入不一致的例子

- 购物车是允许“写入不一致”现象的一个经典示例
 - 即使网络有故障，也总是能够修改购物车中的商品。
 - 这么做有可能导致多个购物车出现
 - 而结账过程则会将两个购物车合并，具体做法是，将两个购物车中的每件商品都拿出来，放到另外一个购物车中，并按照新的购物车结账。
 - 这个办法基本上不会出错，万一有问题，客户也有机会在下单之前先检视一下购物车中的东西。

BASE

- 与关系型数据库所支持的ACID事务不同，NoSQL系统具备“BASE属性”
 - 基本可用，Basically Available
 - 柔性状态，Soft state
 - 最终一致性，Eventual consistency
- “ACID”与“BASE”不是非此即彼的关系，两者之间存在着多个逐渐过渡的权衡方案可选。

“一致性”与“延迟”之间取舍

- 在权衡分布式数据库的“一致性”时，与其考虑如何权衡“一致性”与“可用性”，不如思考怎样在“一致性”与“延迟”(latency)之间取舍。
 - 参与交互操作的节点越多，“一致性”就越好
 - 然而，每新增一个节点，都会使交互操作的响应时间变长
 - “可用性”可以视为能够忍受的最大延迟时间，一旦延迟过高，我们就放弃操作，并认为数据不可用
 - 这样一来，就和“CAP定理”对“可用性”所下的定义相当吻合了

“持久性”的权衡

- 数据库大部分时间都在内存中运行，更新操作也直接写入内存，并且定期将数据变更写回磁盘
 - 可以大大提高响应请求的速度。
 - 代价在于，一旦服务器发生故障，任何尚未写回磁盘的更新数据都将丢失。
- 多用户的“会话状态”信息
 - 会话数据就算丢失，与应用系统效率相比，也不过是个小麻烦。这时可以考虑非持久性写入操作”(nondurable write)。
 - 可以在每次发出请求时，指定该请求所需的持久性。从而，把某些极为重要的更新操作立刻写回磁盘。
- 捕获物理设备的遥测数据(telemetric data)。就算最近的更新数据可能会因为服务器发生故障而丢失，也还是选择把快速捕获数据放在首位。

分布模型中“持久性”的权衡 1

- 如一个节点处理完更新操作之后，在更新数据尚未复制到其他节点之前就出错了，那么则会发生“复制持久性”(replication durability) 故障。
- 假设有一个采用“主从式分布模型”的数据库，在其主节点出错时，它会自动指派一个从节点作为新的主节点。
 - 若主节点发生故障，则所有还未复制到其他副本的写入操作就都将丢失
 - 一旦主节点从故障中恢复过来，那么，该节点上的更新数据就会和发生故障这段时间内新产生的那些更新数据相冲突
 - 我们把这视为一个“持久化”问题，因为主节点既然已经接纳了这个更新操作，那么用户自然就会认为该操作已经顺利执行完，但实际上，这份更新数据却因为主节点出错而丢失了

分布模型中“持久性”的权衡 2

- 解决方案：
 - 不重新指派新的主节点
 - ◆ 在主节点出错之后迅速将其恢复
 - 确保主节点在收到某些副本对更新数据的确认之后，再告知用户它已接纳此更新
 - ◆ 从节点发生故障时，集群不可用
 - ◆ 拖慢更新速度
- 与处理“持久性”的基本手段类似，也可以针对单个请求来指定其所需的持久性。



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch69 仲裁

Software Institute
Nanjing University
Bei Jia

写入仲裁

- 处理请求所节点越多，避免“不一致”问题的能力就越强，要想保“强一致性”(strong consistency)，需要使用多少个节点才行？
- “对等式分布模型”：
 - “写入仲裁”(write quorum)：如果发生两个相互冲突的写入操作，那么只有其中一个操作能为超过半数的节点所认可， $W > N/2$ 。即，参与写入操作的节点数(W)，必须超过副本节点数(N)的一半。副本个数又称为“复制因子”
- “主从式分布模型”
 - 只需要向主节点中写入数据

读取仲裁

- 想要保证能够读到最新数据，必须与多少个节点联系才行？
- “对等式分布模型”：
 - 只有当 $R+W>N$ 时，才能保证读取操作的“强一致性”。其中，执行读取操作时所需联系的节点数(R)，确认写入操作时所需征询的节点数(W)，以及复制因子(N)
- “主从式分布模型”
 - 只需从主节点中读取数据

复制因子

- “复制因子”(replication factor)。
 - 一个集群有100 个节点，然而其“复制因子”可能仅仅是3，因为大部分数据都分布在各个“分片”之中。
- 将“复制因子”设为3，就可以获得足够好的“故障恢复能力”了。
 - 如果只有一个节点出错，那么仍然能够满足读取与写入所需的最小法定节点数。
 - 若是有自动均衡(automatic rebalancing) 机制，那么用不了多久，集群中就会建立起第三个副本，在替代副本建立好之前，再次发生副本故障的概率很小。

实际情况

- 需要在“一致性”与“可用性”之间权衡，参与某个操作的节点数，可能会随着该操作的具体情况而改变。
 - 在写入数据时，根据“一致性”与“可用性”这两个因素的重要程度，有一些更新操作可能需要获得足够的节点支持率才能执行，而另外一些则不需要。
 - 与之相似，某些读取操作可能更看中执行速度，而可以容忍过时数据，此时，它就可以少联系几个节点。
- 通常需要协调考虑读、写两种情况：
 - 假设需要快速且具备“强一致性”的读取操作，那么写入操作就要得到全部节点的确认才行，这样的话，只需联系一个节点，就能完成读取操作了 ($N=3, W=3, R=1$)
 - 但是，这个方案意味着，写入操作会比较慢，因为它们必须得到全部三个节点确认之后，才能执行，而且此时连一个节点都不能出错。



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch70 版本戳

Software Institute
Nanjing University
Bei Jia

商业活动

- “商业活动”(Business Activity).
 - 例如，用户浏览产品目录，选中了一瓶价格很实惠的Talisker威士忌，填入信用卡信息，然后确认订单。
- 需要确保最终一致性，但是出于时间、交互的考虑，无法使用事务加以实现
 - 如使用事务实现，必须锁住数据库中各个元素。而长时间锁定元素是不现实的。
 - 因此，应用程序通常只在处理完用户交互操作之后才开始“系统事务”，这样的话，锁定时间就比较短了。
 - 然而当需要计算和决策的时候，数据有可能已经改动了。
 - ◆ 价格表上Talisker威士忌的售价也许已经变了，或是某人可能会修改客户的地址，从而导致运费改变

条件更新和版本戳

- “条件更新”(conditional update), 客户端执行操作时, 将重新读取“商业活动”所依赖的信息, 并检测该信息在首次读取之后是否一直没有变动, 若一直未变, 则将其展示给用户。
- 通过保证数据库中的记录都有某种形式的“版本戳”(version stamp) 实现“乐观离线锁”(Optimistic Offline Lock) 。
 - 版本戳是一个字段, 每当记录中的底层数据改变时, 其值也随之改变。
 - 读取数据时可以记下版本戳, 这样的话, 在写入数据之前, 就可以先检查一下数据版本是否已经变了。
- 使用版本戳避免“更新冲突”, 维护“会话一致性”
- “CAS”操作 (“compare-and-set”操作)
 - 既可以由数据库提供
 - 也可以由开发者负责检测的执行

构建版本戳的方法 1

- 使用计数器
 - 每当资源更新时，就把它的值加1。根据它的值很容易就能看出哪个版本比较新。
 - 需要服务器来生成该值，并且要有一个主节点来保证不同版本的计数器值不会重复。
- 使用GUID（全局唯一标识符），也就是一个值很大且保证唯一的随机数。
 - 可以将日期、硬件信息，以及其他一些随机出现的资源组合起来以构建此值。
 - 好处是任何人都可以生成，不用担心重复。
 - 缺点则是数值比较大，而且无法通过直接比较来判断版本的新旧。

构建版本戳的方法 2

- 根据资源内容生成哈希码(hash)。只要哈希键足够大，那么“内容哈希码”(content hash)就可以像GUID那样全局唯一，而且任何人都可以来生成它。
 - 好处在于，哈希码的内容是确定的:只要资源数据相同，那么任何节点生成的“内容哈希码”都是一样的。
 - 但是，哈希码与GUID一样，都无法通过直接比较来看出版本新旧，而且比较冗长。
- 使用上一次更新时的时间戳(timestamp)。
 - 与计数器一样，时间戳也相当短小，而且可以直接通过比较其数值来确定版本先后
 - 时间戳不需要由主节点来生成，可以由多台时钟同步的计算机生成。如果某个节点的时钟出错了，那么可能会导致各种数据损毁(data corruption)现象。
 - 若时间戳精确度过低，则可能重复：假如每毫秒都要更新很多次的话，那么将时间戳的精确度设为毫秒是不够的。

构建版本戳的方法 3

- 可以把几种时间戳生成方案的优点融合起来，同时使用多种手法创建一个“复合版本戳”(composite stamp)。
- 在CouchDB创建版本戳时，使用了计数器与“内容哈希码”。
 - 大部分情况下，只要比较版本戳就可以判定两个版本的新旧
 - 万一碰到两个节点同时更新数据的情况，因为两个版本戳的计数器相同，而“内容哈希码”却不同，立刻就能发现冲突。

“主从式复制模型”中的版本戳

- 在“主从式复制模型”中，只有一个权威数据源 (authoritative source for data)，使用基本的版本戳生成方案
- 由主节点负责生成版本戳，而从节点必须使用主节点版本戳。
 - 以计数器为例，节点每次更新数据时，都将它加1,并把其值放入版本戳中。
 - 假设某主节点有两个副本，分别是“蓝色”节点和“绿色”节点。
 - 如果在蓝色节点所给出的应答数据中，版本戳为4，而绿色节点的版本戳是6，那么绿色节点上的数据就比较新。

“对等式分布模型”中的版本戳 1

- 在“对等式分布模型”中，没有统一设置版本戳的节点
- 如果向两个节点索要同一份数据，那么有可能获得不同的答案
 - 有可能是更新操作已经通知给其中一个节点了，而另外一个节点尚未收到通知
 - ◆ 可以选用最新的数据
 - 发生了“更新不一致”现象

“对等式分布模型”中的版本戳 2

- 在“对等式分布模型”中

- 确保所有节点都有一份“版本戳记录”(version stamp history)。从而判断出蓝色节点给出的应答数据是不是绿色节点所给数据的“祖先”。

- ◆ 要么让客户端保存“版本戳记录”，要么由服务器节点来维护此记录，并且把它放在应答数据中，传给客户端。

- ◆ 用“版本戳记录”可以检测出数据“不一致”现象

- 如果两份应答数据中的版本戳都无法在对方的“版本戳记录”中找到，那么就可以判定发生了“不一致”问题。

- 使用“时间戳”

- ◆ 很难确保所有节点的时间都一致

- ◆ 无法检测“写入冲突”

数组式版本戳 1

- “数组式版本戳” (vector stamp) 由一系列计数器组成，每个计数器都代表一个节点。
- 假设有三个节点(分别记为“蓝色(blue)”、“绿色”(green)、“黑色”(black)),那么一个可能的“数组式版本戳” 就类似[blue: 43, green :54, black: 12] 。
- 每当节点执行“内部更新”(internal update)操作时，就将其计数器加1，
 - 假设绿色节点执行了一次更新操作，那么现在这个“数组式版本戳”就成了[blue: 43, green: 55, black: 12]。
- 只要两个节点通信，它们就同步其“数组式版本戳”。具体的同步方式有很多种。
- 使用此方案，就能辨别某个“数组式版本戳”是否比另外一个新，因为版本戳中的计数器总是大于或等于旧版本戳。
 - 比如， [blue: 1, green: 2, black: 5]就比[blue: 1, green: 1, black: 5]新

数组式版本戳 2

- 若两个版本戳中都有一个计数器比对方大，那么就发生了“写入冲突”
 - 比如，[blue: 1, green: 2, black: 5] 与 [blue: 2, green: 1, black: 5] 相冲突
- 数组中可能缺失某些值，我们将其视为0。
 - 比如，[blue: 6, black: 2] 与 [blue: 6, green: 0, black: 2] 等价。
 - 不需要弃用现有的“数组式版本戳”，就可以向其中轻易新增节点。
- “数组式版本戳”是一种能够侦测出“不一致”现象的有用工具，然而它们无法解决此问题。要想解决冲突，就得依赖领域知识。
- 在“一致性”与延迟之间权衡时。
 - 如果偏向“一致性”，那么系统在出现“网络分区”现象时就无法使用
 - 反之，若要减少延迟，则必须自己检测并处理“不一致”问题。